

VibeBench: An Automated Framework for the Holistic Evaluation of LLM-Generated Code

Muktadir Arif ¹

¹ Saint Joseph Higher Secondary School, Dhaka, Bangladesh

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

The rapid integration of Large Language Models (LLMs) into the software development lifecycle has created a critical need for evaluation frameworks that extend beyond basic functional correctness. Traditional benchmarks, such as HumanEval or MBPP, primarily measure a model's ability to pass unit tests (functional correctness). However, they often overlook essential software engineering attributes like code maintainability, structural complexity, and operational resource efficiency.

VibeBench is an automated, extensible Python framework designed to fill this gap. It provides a holistic evaluation of LLM-generated code by integrating static quality heuristics with sandboxed dynamic execution. By walking the Abstract Syntax Tree (AST), VibeBench quantifies Halstead complexity metrics and identifies non-standard “bad practices”—such as ghost comments, hardcoded credentials, and documentation gaps—that are frequently found in AI-synthesized outputs but missed by traditional testers.

Simultaneously, the framework manages a secure lifecycle for generated scripts using Unix-based resource limiting (CPU time and memory address space) to measure runtime stability and latency. VibeBench is built for AI researchers, software auditors, and developers who need to quantify the “technical debt” introduced by autonomous agents. By grounding AI performance against a formalized Human Baseline, it provides the necessary metrics to determine if AI-generated code is truly production-ready.

VibeBench is designed for three primary user groups. AI researchers studying model evolution can use it to conduct longitudinal studies comparing how code quality changes across model versions. Software engineering teams evaluating AI coding assistants for production deployment can use it to audit technical debt before it accumulates. Benchmark designers can use it as a foundation for extending evaluation beyond functional correctness. The framework is released under the MIT License and designed to be extensible — new models, tasks, and heuristics can be added without modifying the core pipeline.

Statement of need

Current evaluation methodologies for Large Language Model (LLM) generated code, such as HumanEval ([Chen & others, 2021](#)) and MBPP ([Austin & others, 2021](#)), primarily focus on functional correctness through unit testing. While these benchmarks are effective at determining if a model can solve a specific problem, they fail to audit the structural integrity and production-readiness of the resulting code. In a production environment, code that “works” but is overly complex, undocumented, or resource-inefficient creates significant technical debt and security risks.

VibeBench addresses this gap by providing a toolset that treats AI-generated code as software artifacts rather than just mathematical solutions. There is a documented “documentation crisis” in LLM outputs, where models frequently omit docstrings and internal comments required for

maintainability. Furthermore, existing benchmarks rarely account for the correlation between high structural complexity and runtime instability.

VibeBench allows researchers and software auditors to:

- **Quantify Technical Debt:** Measure Halstead complexity and documentation coverage to predict long-term maintenance costs.
- **Audit Security and Best Practices:** Detect “AI-isms” like ghost comments or hardcoded credentials using AST-based heuristics.
- **Benchmark Operational Parity:** Compare AI performance against a formalized human baseline in a resource-constrained sandbox.
- **Estimate Environmental Cost:** Compute per-execution carbon footprint estimates to support Green IT research comparing the energy efficiency of AI-generated versus human-authored algorithms.

By providing these capabilities in an open-source, modular format, VibeBench empowers researchers to conduct large-scale longitudinal studies on model evolution, benchmark the energy efficiency of AI-generated algorithms for Green IT research, and build “AI-Audit” pipelines that ensure autonomous agents adhere to human-centric documentation and security standards. This enables the scientific community to move toward a more holistic understanding of AI performance that prioritizes long-term software sustainability over short-term functional success.

State of the field

Several benchmarks and frameworks currently exist for evaluating the code generation capabilities of Large Language Models (LLMs). The most prominent among these are HumanEval (Chen & others, 2021) and MBPP (Mostly Basic Python Problems) (Austin & others, 2021). These benchmarks establish functional correctness by measuring the pass@k metric—a statistical representation of whether a model can produce at least one solution that passes a provided suite of unit tests. Other tools, such as CodeSearchNet (Husain & others, 2019), provide large-scale datasets for code retrieval and summarization but are not designed for direct execution-based benchmarking.

Despite their widespread adoption, these tools share a common limitation: they treat code as a mathematical solution rather than a software artifact. They prioritize “binary success” (the code runs) over “structural health” (the code is maintainable). For instance, HumanEval does not penalize models for producing “spaghetti code” or failing to include docstrings, provided the output passes the unit tests.

VibeBench was developed to bridge this gap between functional testing and software engineering audits. While existing tools like Evaluating Large Language Models Trained on Code focus on the logic of the algorithm, VibeBench provides an automated pipeline for quantifying technical debt. To the authors’ knowledge, VibeBench is among the first frameworks to integrate AST-based heuristic detection for “AI-isms” with Unix-controlled dynamic resource limiting to audit the operational parity of LLM-synthesized software against a formalized human baseline.

Existing static analysis tools such as Pylint and Bandit focus on general Python code quality rather than on patterns specific to LLM-generated outputs, and do not integrate dynamic sandboxed execution or comparison against a human baseline.

Software design

VibeBench is designed as a modular, extensible pipeline written in Python. The framework follows a “Collector-Analyzer-Executor” architecture to ensure that static heuristics and dynamic performance metrics are decoupled and independently verifiable. This separation is deliberate: static analysis operates on source code as text, requiring no execution environment, while dynamic execution requires a controlled subprocess environment with resource limits. By

90 keeping these two analytical tracks independent, VibeBench allows researchers to run static
91 analysis on any Python file regardless of whether it is safe to execute, and to add new heuristics
92 to either track without affecting the other. The core logic is divided into three primary
93 sub-packages, each with a single well-defined responsibility.

94 VibeBench is self-auditing: running `vibebench analyze` against its own source files confirms
95 that all core modules achieve 100% docstring coverage and zero bad practice detections,
96 meeting the standards it applies to evaluated code.

97 To support reproducibility analysis, VibeBench optionally executes each file `N` times via the
98 `--runs N` flag, reporting mean, standard deviation, minimum, and maximum execution time
99 across runs. This allows researchers to distinguish genuine performance differences from
100 single-measurement noise.

101 VibeBench is self-auditing: running `vibebench analyze` against its own source files confirms
102 that all seven core modules achieve 100% docstring coverage, zero bad practice detections,
103 and average cyclomatic complexity of 5.72 — below the McCabe high-risk threshold of 10 and
104 lower than four of the six AI models evaluated in the benchmark. Results are documented in
105 `docs/self-audit.md`.

106 Static Quality Analyzer (`core/analyzer.py`)

107 The `CodeAnalyzer` class serves as the static analysis engine, utilizing Python's built-in `ast`
108 module to parse generated code into a tree structure without execution.

- 109 ▪ **Heuristic Engine:** Implements custom AST visitors to detect “AI-isms” — patterns
110 common in LLM outputs such as ghost comments (empty `#` symbols) and hardcoded
111 credentials.
- 112 ▪ **Complexity Metrics:** Integrates the `radon` and `halstead` libraries to compute Cyclomatic
113 Complexity and Halstead Volume, providing a mathematical basis for maintainability
114 assessment.

115 Sandboxed Dynamic Executor (`core/executor.py`)

116 The `CodeExecutor` implements a secure lifecycle management system for safely evaluating
117 unverified AI-generated code.

- 118 ▪ **Resource Limiting:** Leverages the Unix resource module to enforce hard limits on
119 CPU time (`RLIMIT_CPU`) and maximum memory address space (`RLIMIT_AS`), preventing
120 infinite loops or memory exhaustion from destabilizing the host system.
- 121 ▪ **Isolation:** Each test run executes in a clean environment, capturing `stdout`, `stderr`, and
122 exit codes to determine runtime stability.

123 Beyond timing and correctness, the executor computes an order-of-magnitude carbon footprint
124 estimate for each execution using the formula $CO_2e = t \times P \times I / 3,600,000$, where `t` is
125 execution time in seconds, `P` = 15W is a typical laptop CPU TDP, and `I` = 475 gCO₂/kWh
126 is the IEA 2023 global average carbon intensity. This makes VibeBench among the first
127 code evaluation frameworks to report the environmental cost of AI-generated code execution,
128 supporting Green IT research applications. To support reproducibility, VibeBench optionally
129 executes each file `N` times via the `--runs N` flag, reporting mean execution time, standard
130 deviation (Bessel-corrected), minimum, and maximum across runs — allowing researchers to
131 distinguish genuine performance differences from single-measurement noise.

132 Reporting and Visualization (`core/reporter.py`)

133 The reporting layer aggregates JSON-formatted raw data into human-readable outputs.

- 134 ▪ **Leaderboard Generation:** Automatically calculates averages across multiple trials and
135 generates Markdown tables for direct inclusion in documentation.

136 ▪ **Performance Plotting:** Utilizes `matplotlib` to visualize the correlation between structural
137 complexity scores and execution success rates.

138 Three generator modules (`core/gemini_generator.py`, `core/groq_generator.py`, and
139 `core/openai_generator.py`) provide programmatic code generation via the Google Gemini,
140 Groq, and OpenAI APIs respectively, enabling fully automated benchmark dataset construction
141 across multiple providers.

142 Research impact statement

143 VibeBench enables a category of empirical software engineering research that existing
144 benchmarks cannot support: the systematic audit of LLM-generated code as a software
145 artifact rather than a mathematical solution. By integrating static quality heuristics with
146 sandboxed dynamic execution, VibeBench allows researchers to answer questions that pass/fail
147 unit testing cannot address.

148 In our initial evaluation across six models and five tasks, VibeBench revealed several findings
149 with direct implications for AI-assisted software development. First, all evaluated models
150 exhibited a significant documentation gap: Claude produced 0% docstring coverage across all
151 tasks despite an 80% functional success rate, demonstrating that functional correctness and
152 code maintainability are independent dimensions that must be measured separately. Second,
153 human-authored baseline solutions achieved lower average cyclomatic complexity (3.60) than
154 every evaluated AI model, confirming a systematic over-engineering tendency in LLM outputs
155 that has practical consequences for long-term code maintenance costs. Third, VibeBench's
156 heuristic detection identified a mutable default argument anti-pattern in DeepSeek's TASK-005
157 output — a runtime-risk pattern that caused an actual execution failure and that no existing
158 benchmark would surface.

159 These findings demonstrate that VibeBench fills a genuine measurement gap in the LLM
160 evaluation landscape. Researchers studying model evolution, comparing code generation
161 approaches, or building AI-audit pipelines for production deployment can use VibeBench to
162 quantify dimensions of code quality that are invisible to correctness-only benchmarks. The
163 framework is designed to be extensible, allowing the research community to add new heuristics,
164 models, and task categories as the field evolves.

165 Mathematics

166 VibeBench quantifies software quality through two complementary metric families: static
167 complexity measures derived from source structure, and dynamic operational metrics derived
168 from sandboxed execution.

169 Static Complexity

170 Halstead Volume (V) measures the information content of a program based on operator and
171 operand counts (Halstead, 1977):

$$V = (N_1 + N_2) \cdot \log_2(n_1 + n_2)$$

172 where N_1 and N_2 are the total counts of operators and operands respectively, and n_1 and n_2
173 are the counts of unique operators and operands. Higher volume indicates greater cognitive
174 load required to understand the code.

175 Cyclomatic Complexity (M) quantifies the number of linearly independent paths through a
176 program's control flow graph (McCabe, 1976):

$$M = E - N + 2P \tag{1}$$

177 where E is the number of edges, N is the number of nodes in the control flow graph, and P is
178 the number of connected components. VibeBench flags code with $M > 10$ as high-risk for
179 maintainability failure, consistent with McCabe's original threshold.

180 Dynamic Operational Metrics

181 Operational Parity (Φ) measures how closely an LLM-generated solution matches the runtime
182 efficiency of a human-authored baseline:

$$\Phi = \frac{T_{\text{base}}}{T_{\text{llm}}} \quad (2)$$

183 where T_{base} is the execution latency of the human baseline and T_{llm} is the execution latency
184 of the LLM-generated code under identical resource constraints. A value of $\Phi \approx 1$ indicates
185 optimal parity; $\Phi \gg 1$ indicates the LLM solution is significantly slower than the human
186 baseline.

187 The composite **VibeBench Score** (Σ) aggregates static and dynamic performance into a single
188 normalized metric:

$$\Sigma = w_1 \cdot \hat{V} + w_2 \cdot \hat{M} + w_3 \cdot \Phi$$

189 where $\hat{V}$ and $\hat{M}$ are min-max normalized Halstead Volume and Cyclomatic Complexity
190 respectively, and w_1, w_2, w_3 are configurable weights summing to 1. Default weights are
191 set empirically at $w_1 = 0.4$, $w_2 = 0.4$, $w_3 = 0.2$.

192 Limitations

193 VibeBench is subject to several limitations that constrain the generalisability of its current
194 findings and should be considered when interpreting results.

195 First, the benchmark dataset in its current form covers five tasks across three difficulty levels
196 and six models. While sufficient to demonstrate proof-of-concept findings, this scale is modest
197 compared to established benchmarks such as HumanEval (164 tasks) or MBPP (374 tasks).
198 Conclusions about systematic LLM behaviour should therefore be treated as preliminary until
199 validated across a larger task set.

200 Second, VibeBench currently supports only Python. The “documentation crisis” and
201 over-engineering patterns observed in this study may differ substantially in statically-typed
202 languages such as Java or TypeScript, where type annotations provide an alternative form of
203 documentation and compilers enforce structural constraints that Python does not.

204 Third, the sandboxed execution environment uses Unix-based resource limits (RLIMIT_CPU
205 and RLIMIT_AS) and is therefore not directly portable to Windows without modification. The
206 static analysis components function cross-platform, but dynamic execution results may vary
207 across operating systems.

208 Fourth, the Human Baseline used in this study consists of solutions authored by a single
209 developer. A more robust baseline would aggregate solutions from multiple experienced
210 developers to account for individual variation in coding style and complexity.

211 These limitations represent concrete directions for future work. Expanding the task set, adding
212 multi-language support, and recruiting multiple baseline authors are planned for subsequent
213 releases of VibeBench.

214 **Figures**

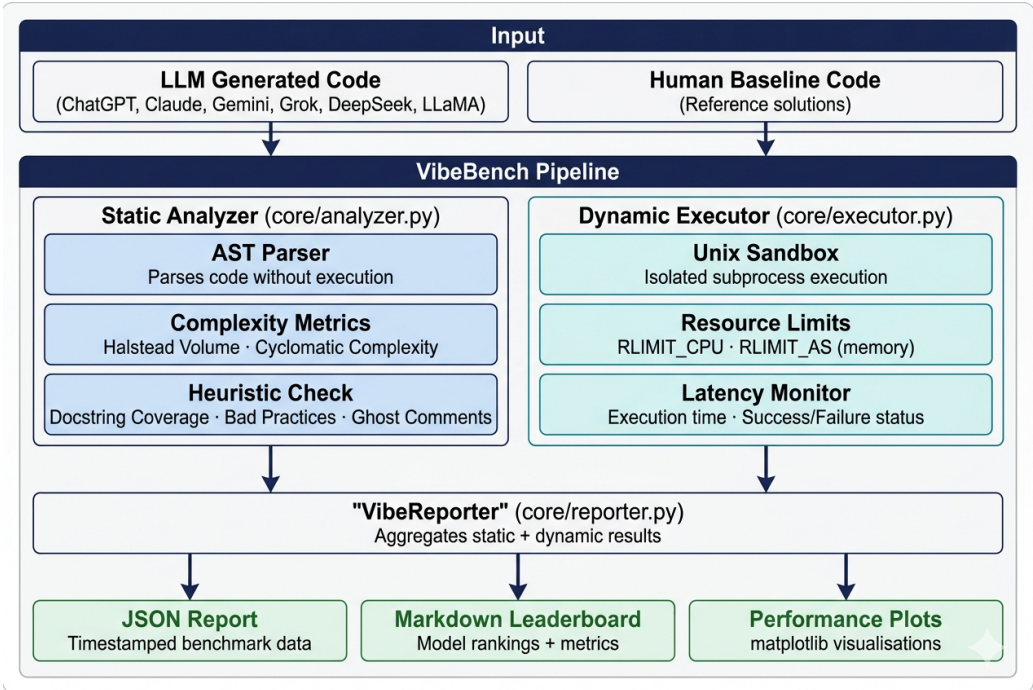


Figure 1: The VibeBench System Architecture, illustrating the modular flow from LLM code ingestion to AST-based static analysis and sandboxed execution.

215 As shown in [Figure 1](#), the framework ensures that static heuristics (Halstead complexity,
216 docstring coverage) are captured independently of dynamic performance metrics.

Rank	Model	Avg Complexity	Avg Exec Time	Avg Doc Coverage	Bad Practices	Success Rate	Total CO ₂ e (μg)
—	HUMAN_SAMPLES	3.7	0.3202s	100.0%	0	10/10	6337.095
1	GEMINI	4.15	0.0844s	87.5%	0	10/10	1669.427
2	CHATGPT	4.28	0.1079s	65.0%	0	10/10	2135.322
3	CLAUDE	3.78	0.0844s	20.0%	1	8/10	1670.219
4	LLAMA_3_3_70B_VERSATILE	5.3	0.0558s	0.0%	1	4/5	551.792
5	DEEPSEEK	5.06	0.085s	94.1%	2	7/10	1683.084
6	GROK	5.32	0.0932s	91.0%	0	6/10	1845.176

Figure 2: The VibeBench Leaderboard showing aggregate performance of seven evaluated systems across ten benchmark tasks. Models are ranked by success rate. Metrics include average cyclomatic complexity, execution time, docstring coverage, bad practice count, and estimated carbon footprint (CO₂e).

217 **AI usage disclosure**

218 In accordance with JOSS policies, the author discloses that generative AI tools (specifically
219 Google Gemini and ChatGPT) were utilized during the development of this project.

- 220 ▪ **Software Development:** AI was used to assist in writing standard boilerplate code for the
221 reporting engine and for debugging Abstract Syntax Tree (AST) visitor patterns. The
222 core logic of the VibeBench evaluation framework, including the specific static quality

heuristics and the dynamic resource-limiting implementation, was designed and verified by the author.

- **Manuscript Preparation:** AI tools were used for language refinement, grammatical correction, and optimizing the LaTeX formatting of the manuscript.

All scientific claims, experimental results, and data interpretations presented in this paper are the original work of the author and have been manually verified for accuracy.

Acknowledgements

The author thanks the faculty and administration of Saint Joseph Higher Secondary School, Dhaka, for fostering an environment that supports independent student research. The author also thanks Mushrat Salehin Nibir for his valuable feedback on the codebase, suggestions for improvement, and for identifying and reporting issues during development. Special thanks are also extended to the Executive Committee and members of the Josephite Scintilla Science Club (JSSC) for their practical feedback on the framework's utility and their continued encouragement of student-led work in software engineering and AI research.

References

- Austin, J., & others. (2021). Program synthesis with large language models. *arXiv Preprint arXiv:2108.07732*.
- Chen, M., & others. (2021). Evaluating large language models trained on code. *arXiv Preprint arXiv:2107.03374*.
- Halstead, M. H. (1977). *Elements of software science*. Elsevier.
- Husain, H., & others. (2019). CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv Preprint arXiv:1909.09436*.
- McCabe, T. J. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, 2(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>